

# Audion Voice AI

Windows 10 | 11

release no releases or repo not found

downloads repo not found

license repo not found

**Version 2.1.1** · 2026-08-24 · 626.2 MB

- [Direct download](#) — unmetered, no rate limits
- [Project page](#) — every version and how to install

SHA-256: f0954cc4916d7f9f03b7202daf6bcd73ee831df85375954230ff171c0adb2e58

Audion Voice AI is a Windows-first desktop app for file transcription, live dictation, text cleanup, and export to a practical knowledge workflow. The product is built around two paths: a fast API workflow for daily work and local GPU/CPU engines for private or long-running transcription jobs.

## Installing the recognition models

The build ships without model weights: the GigaAM cache and the whisper.cpp pack come to nearly five gigabytes together. Weights carry their own licences, apart from the licence of this program, so they are downloaded by you — under your own account where that is required.

Until the models are installed, transcription will not run: the window opens, but there is nothing behind it to recognise speech.

Run `builder_main.cmd` and pick, in order:

| #  | Menu entry               | What it installs                          |
|----|--------------------------|-------------------------------------------|
| 08 | GIGAAM ONNX              | the main Russian speech recognition model |
| 09 | WHISPER.CPP CPU FALLBACK | the CPU fallback engine                   |

Both steps are required. The download comes from HuggingFace and takes as long as several gigabytes usually take.

## FFmpeg and your NVIDIA driver

Newer is not always better. Every FFmpeg build is compiled against one specific version of the NVENC headers, and each of those demands a minimum driver. Put the newest build on an older driver and hardware encoding does not get faster — it stops working.

| FFmpeg build | NVENC headers        | Minimum NVIDIA driver (Windows) |
|--------------|----------------------|---------------------------------|
| 9.0.1        | ffnvcodec n13.1.15.0 | <b>610.0</b>                    |
| 8.0.1        | ffnvcodec n13.0.19.0 | <b>570.0</b>                    |
| 7.1.1        | ffnvcodec n13.0.19.0 | <b>570.0</b>                    |
| 7.1          | ffnvcodec n12.2.72.0 | 551.76                          |

Note the third row: 7.1.1 is built with the same headers as 8.0.1, so it needs the same 570.0 — going "one version back" buys nothing on an older driver. The step that does help is 7.1 without the patch release.

This is why the installer picks a build from your driver version instead of always taking the latest. The versions above are read from the build's own README; the driver thresholds come from the nv-codec-headers README.

If you have no NVIDIA GPU, none of this applies — the latest build is installed and encoding runs on the CPU.

**Which build ships with this product: 8.0.1.** That is a deliberate choice, not a missed update. Most editing and encoding machines today run drivers roughly between 571 and 609; the 610 branch is installed by very few. Both 8.1.x and 9.x demand that branch — shipping them would advertise NVIDIA hardware encoding and then deny it to most of the people it was promised to. 8.0.1 has everything these products use and runs on the drivers people actually have.

## Editions

| Edition                | Purpose                                           | Engines                                      |
|------------------------|---------------------------------------------------|----------------------------------------------|
| Audion Voice AI Live   | Lightweight API Live and local model distribution | OpenAI, xAI, ElevenLabs, GigaAM, whisper.cpp |
| Audion Voice AI Studio | Full workstation distribution for CUDA machines   | API providers, GigaAM, whisper.cpp, CUDA     |

Live is the primary build for laptops and everyday transcription. Studio adds CUDA/faster-whisper, PyTorch, GPU diarization, and large local models for NVIDIA workstations.

## Main Features

- Transcribes audio and video files through OpenAI, local models, or CUDA faster-whisper in Studio.
- Manages a file queue and saves results next to the source file by default.
- Supports live dictation through API models (OpenAI, xAI, ElevenLabs) and Local Models (GigaAM, whisper.cpp).
- Shows a compact live partials overlay over other windows.
- Cleans long live dictation sessions with a selected OpenAI model and custom prompt.
- Exports to Markdown, TXT, JSON, SRT, and WebVTT.
- Sends results to Notion and Obsidian from the GUI and tray actions.
- Persists theme, app language, checkboxes, filled fields, model lists, and workflow settings after restart.
- Includes a Maintenance tab for runtimes, payloads, models, and progress; Reset App lives under Settings.

## Interface

The GUI is built with PySide6 and uses a dense HUD-style layout with two strong themes.

- **Live** - dictation controls, API/Local source selection, provider/mode settings, overlay, and live cleanup.
- **Files** - queue, transcription engine, recording language, OpenAI profile or local model, post-processing/cleanup, subtitles, and export.

- **Settings** - theme, app language, tray behavior, Notion/Obsidian integrations, and global app options.
- **Setup** - recommended install profile, module checks, runtime/payload installation, progress logs, and Reset App.

Themes:

- **Mess Blue** - blue graphite HUD style with light blue accents.
- **Graphite Code** - graphite theme with dark orange accents.

## Quick Start

---

1. Run `builder_main.cmd` for the first portable runtime build, or open an already built app.
2. In the GUI, use the **Setup** tab: the top card shows the detected GPU, recommended profile, and **Recommended** / **Optional** / **Not needed** labels.
3. Check `config\api_key_*.txt` if you use OpenAI, xAI, or ElevenLabs.
4. On first GUI launch, Audion checks and, when needed, installs Live dependencies from the bundled wheel cache. The Live dependencies row in **Maintenance** remains available for manual repair.
5. The microphone check tries the Windows default recording device first and a separate default communications device next. Native 44.1/48 kHz webcam inputs are supported through internal rate conversion; no audio is saved.
6. In Studio, additionally install faster-whisper, CUDA/pyannote, and Large models only when comparative quality tests need them.
7. Add files to the queue, choose an engine, and start processing.

Rows marked **Not needed** are dimmed but still clickable for manual repair scenarios. Restore rows sit at the bottom and are only for restoring the GigaAM provider after package conflicts.

## Engines

---

### API models

OpenAI Live does not expose a raw model catalog: Realtime uses `gpt-realtime-whisper`, and batch fallback uses `gpt-4o-mini-transcribe`. For files, OpenAI uses intent profiles instead of raw model IDs: **Fast / economical** (`gpt-4o-mini-transcribe`), **Max accuracy** (`gpt-4o-transcribe`), or **With diarization** (`gpt-4o-transcribe-diarize`). xAI and ElevenLabs are wired as fixed realtime Live providers.

## Local Models

The local workflow lets the user choose between GigaAM and whisper.cpp. In the Russian UI layout, GigaAM is the preferred local model; whisper.cpp is a CPU fallback in Live and a CUDA/cuBLAS GPU pack in Studio. `GigaAM ONNX pack` installs `onnx-asr`, an ONNX Runtime provider, and preloads CTC/RNN-T payloads into `models\huggingface`; Windows auto uses DirectML as the lightweight universal backend, while Studio uses CUDA on NVIDIA. GigaAM Live keeps the model warm until the app exits so the first words are not lost. Lightweight GigaAM/ONNX diarization is planned for Live separately from Studio's heavier CUDA/pyannote path.

Backend packages are installed as follows: the distribution includes `install\wheels\live` for offline microphone setup, while `Dependency wheel cache` builds local wheels for the remaining engines. Live creates `live`, `common`, `directml`, and `cpu`; Studio also creates `cuda`. DirectML needs no external SDK, CPU fallback uses `install\wheels\cpu`, and CUDA is Studio-only on NVIDIA after the driver/runtime is installed. TensorRT is not part of the project profile. See `USER_GUIDE_EN.md` for details.

## CUDA

Available in Studio. CUDA uses PyTorch/faster-whisper on NVIDIA GPUs for large-v2 and large-v3-turbo processing. In the GUI, the `CUDA` card offers `Quality` / `Speed`: `Quality` is the default and keeps regular Faster-Whisper/CTranslate2 for calmer load, a more detailed timeline, and better diarization suitability; `Speed` enables batched inference (`batch_size=16`) for fast long-file runs with higher GPU utilization. Full CUDA smoke tests are expected on a CUDA workstation.

## Files and Formats

---

The queue is intended for common audio and video formats that FFmpeg handles reliably. Exotic containers should be converted before processing. Results are saved next to the source file by default, so users do not need to search through a separate `output` folder.

Export formats:

- Markdown
- TXT
- JSON
- SRT
- WebVTT

## Live Dictation

---

Live dictation can be started with the app, from the GUI, or from the tray. The overlay shows live partials without blocking capture. Its right-click menu opens the latest 20 completed dictations, while tray opens the scrollable 200-entry cache and prunes the oldest records automatically. Click a card to paste it again, or use the round controls to copy or delete it. Overlay height is configured in the **Live** tab. Long sessions can be cleaned automatically after a configured sentence count.

## Tray

---

The tray menu provides quick actions without opening the main window:

Minimizing sends the app to the tray. Closing the window exits the app completely and removes the tray icon.

- start and stop live dictation;
- export the current log to Markdown;
- export materials to Notion or Obsidian;
- access common app actions.

Tray mode can be disabled in Settings.

## Reset App

---

**Reset App** restores default UI and workflow settings. It must not remove API keys, installed runtimes/payloads, downloaded models, or user work files.

## Cleanup and Reproducibility

---

`cleanup_project.cmd` removes everything that can be reproduced on another system: **runtime**, **Tools**, **models**, **install\download**, **install\wheels**, and working payload folders such as **input**, **output**, **logs**, **report**, **workspace**, and **release**. This is intentional: **input** is a temporary working area, not a user archive.

Protected areas are **config**, **Docs**, **tests**, **system\_core**, install scripts, and root launch/build files. After cleanup, the empty structure is recreated through `install\init_folders.cmd`.

## Project Structure

---

```
Audion Voice AI Live/  
  app/  
  config/  
  install/  
  system_core/  
  tools/  
  Docs/  
  builder_main.cmd  
  cleanup_project.cmd
```

Studio mirrors the Live structure and adds CUDA/PyTorch/faster-whisper payloads.

## Documentation

---

- `Docs/USER_GUIDE_RU.md` - Russian user guide.
- `Docs/USER_GUIDE_EN.md` - English user guide.
- `Docs/CODEX_CODE_CHANGES_2026-06-18.md` - implementation change log.
- `Docs/CODEX_PRODUCT_EDITIONS_2026-06-18.md` - Live/Studio edition notes.
- `Docs/CODEX_PLAN_LOCAL_STT_2026-06-18.md` - local STT plan.
- `Docs/CODEX_PLAN_HARDWARE_STT_EDITIONS_2026-06-18.md` - hardware edition plan.

## Current Status

---

Live GUI, settings persistence, overlay, tray actions, API/Local switching, and installer workflow are in a working state. The target local profiles were smoke-tested on RTX 5070: Live GigaAM DirectML, Live whisper.cpp CPU fallback, Studio GigaAM CUDA, and Studio whisper.cpp CUDA/cuBLAS. The GUI module catalog and `builder_main.cmd` are synchronized; TensorRT is excluded from the user-facing profile.

## Acceptance

---

Test one short recording before a long session. Verify the selected edition, engine, language, input device or source file, output text, timestamps when requested, and export format. Keep local and API workflows separate when diagnosing latency or recognition quality.

Live operation should stop cleanly, release the audio device, close overlay/tray helpers on explicit Exit, and preserve a diagnostic log without exposing recognized sensitive content or API keys unnecessarily.